Developer Guide: System Workflow

Portfolio Context

The challenge: The platform's booking engine relies on a strict, multi-step asynchronous workflow (Shopping → Booking → Payment). My task was to bridge the gap between static API schema definitions and the *behavioral* logic required by developers.

My approach: I moved away from "wall-of-text" instructions and modularized the guides. I created structural tables to show exactly which parameters must pass from step to step, paired with annotated XML payloads that explain the *why* behind the required fields.

Note: Elements of the payload and company specifics have been anonymized for this portfolio piece.

Guide: Creating a Booking

Creating a booking is the second required step in any standard transaction flow. It involves sending a `createBookingRecordRequest` message that repeats the key data for whichever fare you selected from the previous shopping response.

You must also provide passenger details and contact information. A successful request places the inventory on hold and establishes a set period (the *Price Guarantee Time*) during which you must complete the payment step.

Request parameters

In its simplest form, the booking request takes the following inputs. Except for passenger data, all necessary information should be extracted directly from your initial shopping response to ensure exact payload matching.

Input Parameter	XML Element
Context headers Every message must include five mandatory identifiers defining your	`<context>`

Input Parameter	XML Element
access scope and authorized features.	
Passenger mapping The names, ages, and metadata of all traveling passengers, plus contact information for at least one lead passenger.	`<passengers>`
Segment solutions A valid routing solution copied precisely from the `shoppingResponse` message.	`<legSolution>`
Pricing blocks Price information for the chosen segment, plus an optional seat reservation request if dynamic assignments are available.	`<prices>`
Transaction behaviors Optional controllers, such as whether to accept a different ticket price if the underlying carrier altered the fare during the booking attempt.	`<parameters>` `<responseSpec>`

Re-shop validation

When you submit the request, the platform performs an automated background re-shop using the `legSolution` data to ensure the inventory is still physically available and the pricing schema hasn't changed before locking the hold.

Annotated Code Example: One-way booking

In this example, we'll look at a simple one-way booking request for a single passenger.

Click the `+` icons to read annotations about specific fields.

createBookingRecordRequest.xml

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:book="http://platform.net/ws/booking"
  xmlns="http://platform.net/ws/commontypes">
  <soapenv:Header/>
  <soapenv:Body>
```

```
    <book:createBookingRecordRequest> <!-- (1)! -->
      <context>
        <distributorCode>DEMO</distributorCode>
        <pointOfSaleCode>US</pointOfSaleCode>
        <channelCode>CON</channelCode>
      </context>
      <book:passengers> <!-- (2)! -->
        <book:passenger passengerID="PAX_SPEC_0">
          <nameFirst>Jane</nameFirst>
          <nameLast>Smith</nameLast>
          <contactInformation> <!-- (3)! -->
            <contact>
              <contactType>BUSINESS</contactType>
              <emailAddress>jane.smith@example.com</emailAddress>
            </contact>
          </contactInformation>
        </book:passenger>
      </book:passengers>
      <book:legSolution> <!-- (4)! -->
        <!-- Omitted for brevity: Exact copy of legSolution from Shopping -->
      </book:legSolution>
    </book:createBookingRecordRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

1. **Required wrapper**: This initiates the hold on the inventory.
2. **Passenger mapping**: The ID (`PAX_SPEC_0`) must be referenced later in the pricing blocks so the system knows which passenger owns which ticket.
3. **Carrier mandates**: Some connected carriers strictly refuse the booking if the `<emailAddress>` block fails regex validation.
4. **State persistence**: The leg solution represents the exact hashed train/bus segment returned during the shopping phase.

Understanding the Response

The resulting `createBookingRecordResponse` message returns three critical states for your application logic:

1. **Success/Fail status**: Indicates if the carrier accepted the hold.
2. **Locator ID**: A unique UUID used to identify the reservation in all subsequent payment/refund calls.
3. **Hold Expiry**: The `holdDateTime` attribute after which the booking will be automatically cancelled.